

AML Presentations (10/03/2019)

Topic 1: Training Set Attack using Machine Teaching

Topic 2: Targeted Clean-Label Attack on Neural Networks

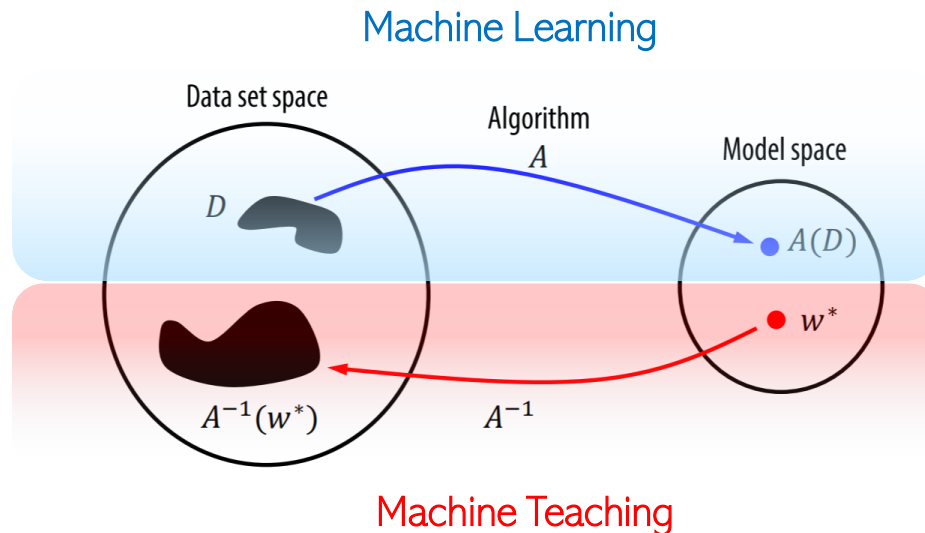
Presenter: Yao Zhou

The logo of the University of Illinois, featuring a stylized orange 'I' followed by the word 'ILLINOIS' in dark blue capital letters.

I ILLINOIS

The Concept of Machine Teaching

- ❑ Machine teaching – an inverse problem of machine learning



W^* : Target concept (e.g., a coefficient vector in Logistic Regression)

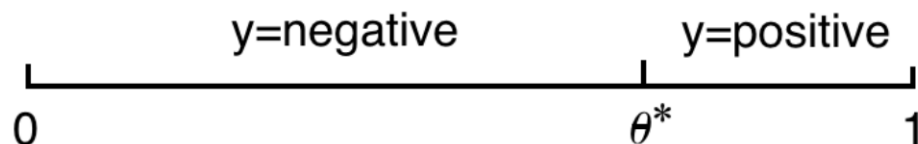
A : Learning algorithm (e.g., gradient descent)

D : Training set (e.g., examples)

The Concept of Machine Teaching

□ A few demonstration examples

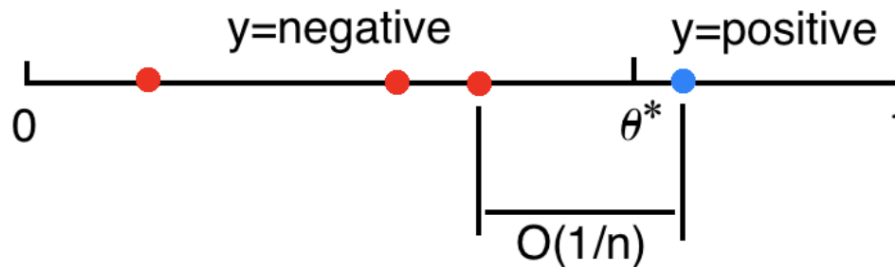
- Consider learning a 1D threshold classifier.
- Input is uniformly distributed on $[0, 1]$.
- The true threshold is θ^* .



The Concept of Machine Teaching

□ A few demonstration examples

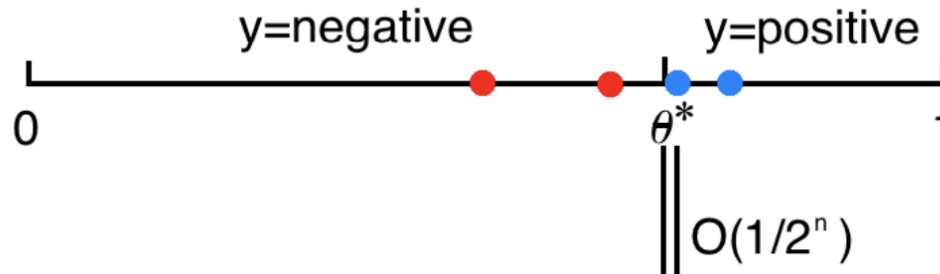
- *Passive learning*
- A consistent learner incurs a generalization error of $|\hat{\theta} - \theta^*| = O(n^{-1})$
- To achieve a ϵ generalization error, it would need $n > O(\epsilon^{-1})$ training items.



The Concept of Machine Teaching

□ A few demonstration examples

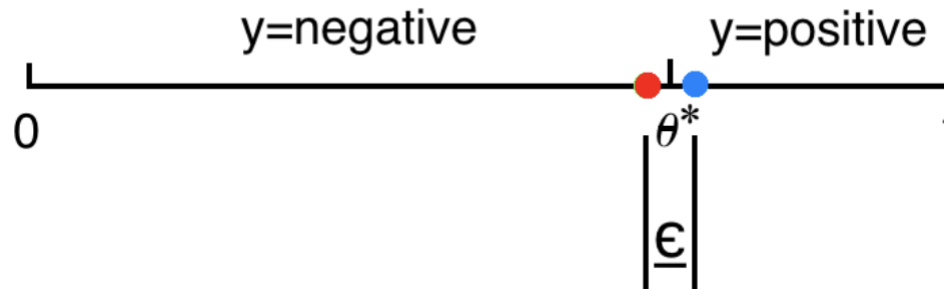
- *Active learning*
- The generalization error is reduced by $|\hat{\theta} - \theta^*| = O(2^{-n})$
- To achieve a ϵ generalization error, it would need $n > O(\log(\epsilon^{-1}))$ queries.



The Concept of Machine Teaching

□ A few demonstration examples

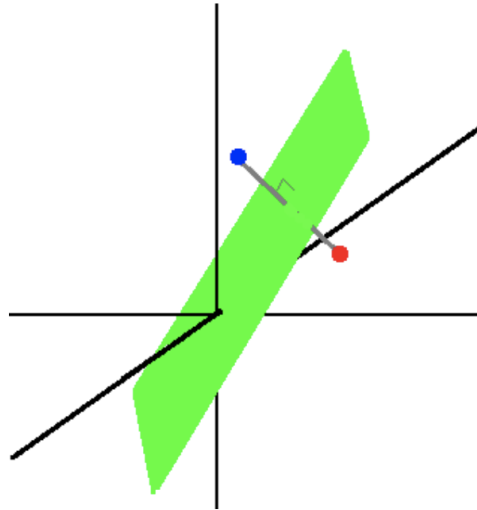
- *Machine teaching*
- Teacher knows θ^* and can construct a set of two items at most ϵ apart and have θ^* in middle
- The teaching set size is 2 regardless the magnitude of ϵ .



The Concept of Machine Teaching

□ Another example: teaching hard margin SVM

- *Teaching dimension*: the smallest teaching set size of this teaching task $TD = 2$



$$TD = 2 \text{ vs. } VC = d + 1$$

The Concept of Machine Teaching

□ Contrast of machine learning (ML) and machine teaching (MT)

- **ML**: the regularized empirical risk minimization framework

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} \sum_{(x,y) \in D} \ell(x, y, \theta) + \lambda \|\theta\|^2$$

- **MT**: teacher knows target model θ^* , and aims to find a teaching set D .

$$\begin{aligned} \min_{D, \hat{\theta}} \quad & \|\hat{\theta} - \theta^*\|^2 + \eta \|D\|_0 \\ \text{s.t.} \quad & \hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} \sum_{(x,y) \in D} \ell(x, y, \theta) + \lambda \|\theta\|^2 \end{aligned}$$

The Concept of Machine Teaching

□ A more general form of teaching

- $\mathbf{R}_T(\cdot)$: teaching risk function, e.g., $\|\hat{\theta} - \theta^*\|_2^2$
- $\mathbf{E}_T(\cdot)$: teaching effort function, e.g. different item cost
- **Teacher's search space**: constructive or pool-based, batch or sequential

$$\begin{array}{ll} \min_{D, \hat{\theta}} & \text{TeachingRisk}(\hat{\theta}) + \eta \text{TeachingCost}(D) \\ \text{s.t.} & \hat{\theta} = \text{MachineLearning}(D). \end{array}$$

The Concept of Machine Teaching

□ Characterize the machine teaching space (teacher **T** teaches student **S**)

- Data Attack!
- **T=machine, S=machine:** E.g., data poisoning attacks.

$$\begin{aligned} \min_{\delta, \hat{\theta}} \quad & \|\hat{\theta} - \theta^*\| + \eta \|\delta\| \\ \text{s.t.} \quad & \hat{\theta} = A(D + \delta) \end{aligned}$$

- Crowd Teaching!
- **T=machine, S=human:** E.g., computer tutoring system, where the key is to assume a cognitive model $\mathcal{A}$ of the human learner.
 - **T=human, S=machine:** E.g., human-in-the-loop intervention of fast training a text classifier. For a 1D threshold classifier, human can produce an optimal teaching set.
 - **T=human, S=human:** E.g. the traditional education in real life..

Using Machine Teaching to Attack Training Set

□ Quick overview of the framework

- **Assumption:** Attacker has full knowledge of the learning algorithm
- **Target:** Poison the training set in order to mislead the learner to a specific wrong models
- **Approach:** Bi-level optimization
- **Solution:** Covert the bi-level optimization to a single level optimization problem using KKT condition
- **Instantiations:** SVM, Logistic Regression, Linear Regression

Using Machine Teaching to Attack Training Set

□ Preliminary

- Machine learner:

$$\begin{aligned} \hat{\theta}_D \in \operatorname{argmin}_{\theta \in \Theta} \quad & O_L(D, \theta) \\ \text{s.t.} \quad & g_i(\theta) \leq 0, \quad i = 1 \dots m \\ & h_i(\theta) = 0, \quad i = 1 \dots p \end{aligned}$$

- **The learner** has a regularized risk minimization objective:

$$O_L(D, \theta) = R_L(D, \theta) + \lambda \Omega(\theta)$$

- **The attacker (teacher)** want to manipulate the original training data D_0 , by changing existing items (x, y) to be (x', y) or (x, y') to get D .

Using Machine Teaching to Attack Training Set

□ Preliminary

- Attacker's risk function:

$$R_A(\hat{\theta}_D) = \|\hat{\theta}_D - \theta^*\|$$

- Attacker's effort function:

$$E_A(D, D_0) = \|X - X_0\|_F$$

- Attacker's overall objective:

$$\begin{array}{ll} \min_{D \in \mathbb{D}, \hat{\theta}_D} & R_A(\hat{\theta}_D) + E_A(D, D_0) \quad \leftarrow \text{upper-level problem} \\ \text{s.t.} & \hat{\theta}_D \in \operatorname{argmin}_{\theta \in \Theta} O_L(D, \theta) \quad \leftarrow \text{lower-level problem} \\ & \text{s.t. } \mathbf{g}(\theta) \leq \mathbf{0}, \mathbf{h}(\theta) = \mathbf{0}. \end{array}$$

Using Machine Teaching to Attack Training Set

□ How to solve it?

- Introduce KKT multiplier for lower-level constraints, the problem becomes:

$$\begin{aligned} \min_{D \in \mathbb{D}, \theta, \lambda, \mu} \quad & O_A(D, \theta) \\ \text{s.t.} \quad & \partial_{\theta} (O_L(D, \theta) + \lambda^{\top} \mathbf{g}(\theta) + \mu^{\top} \mathbf{h}(\theta)) = \mathbf{0} \\ \text{Complementary slackness} \rightarrow & \lambda_i g_i(\theta) = 0, \quad i = 1 \dots m \\ & \mathbf{g}(\theta) \leq \mathbf{0}, \quad \mathbf{h}(\theta) = \mathbf{0}, \quad \lambda \geq \mathbf{0}. \end{aligned}$$

↑ **Primal feasibility** ← **Dual feasibility**

- Using projected gradient descent:

$$D^{(t+1)} = \text{Prj}_{\mathbb{D}} \left(D^{(t)} + \alpha_t \nabla_D O_A(D, \theta^{(t)}) \Big|_{D=D^{(t)}} \right)$$

Where α_t is the step size. Next, we fix $D^{(t+1)}$ and solve for $\theta^{(t+1)}, \lambda^{(t+1)}, \mu^{(t+1)}$

Using Machine Teaching to Attack Training Set

□ What's the issue?

- The gradient of the upper problem is:

$$\nabla_D O_A(D, \theta) = \nabla_{\theta} O_A(D, \theta) \frac{\partial \theta}{\partial D}$$

A Jacobian matrix of size $d \times n$, which does not have an explicit expression

- Use the **implicit function theorem (IFT)** to help.

- Equalities in KKT can introduce a function $\mathbf{f}: \mathbb{R}^{n+d+m+p} \mapsto \mathbb{R}^{d+m+p}$

$$\mathbf{f}(D, \theta, \lambda, \mu) = \begin{pmatrix} \partial_{\theta} (O_L(D, \theta) + \lambda^{\top} \mathbf{g}(\theta) + \mu^{\top} \mathbf{h}(\theta)) \\ \lambda_i g_i(\theta), i = 1 \dots m \\ \mathbf{h}(\theta) \end{pmatrix}$$

- The following solution is induced (if $\mathbf{f}$ is differentiable w.r.t. its parameters and its Jacobian matrix $\left[\frac{\partial \mathbf{f}}{\partial \theta} \middle| \frac{\partial \mathbf{f}}{\partial \lambda} \middle| \frac{\partial \mathbf{f}}{\partial \mu} \right]$ is full-rank) by defining $(\theta, \lambda, \mu) = i(D)$

$$\frac{\partial i}{\partial D} = - \left[\frac{\partial \mathbf{f}}{\partial \theta} \middle| \frac{\partial \mathbf{f}}{\partial \lambda} \middle| \frac{\partial \mathbf{f}}{\partial \mu} \right]^{-1} \left(\frac{\partial \mathbf{f}}{\partial D} \right)$$

- Then, $\frac{\partial \theta}{\partial D}$ is the first d rows of $\frac{\partial i}{\partial D}$

Using Machine Teaching to Attack Training Set

□ Attacking SVM

- Original data set: $D_0 = (\mathbf{X}_0, \mathbf{y}_0)$
- Attack search space: $\mathbb{D} = \{(\mathbf{X}, \mathbf{y}_0) \mid \mathbf{X} \in \mathbb{R}^{d \times n}\}$
- Learner (lower-level problem):

$$\begin{aligned} O_L(D, \mathbf{w}, b, \xi) &= \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i \\ g_i &= 1 - \xi_i - y_i(\mathbf{x}_i^\top \mathbf{w} + b) \\ g_{i+n} &= -\xi_i \end{aligned}$$

- Attacker **risk function** and **effort function** are:

$$R_A(\hat{\mathbf{w}}_D) = \frac{1}{2} \|\hat{\mathbf{w}}_D - \mathbf{w}^*\|_2^2 \quad E_A(D, D_0) = \frac{\lambda}{2} \|\mathbf{X} - \mathbf{X}_0\|_F^2$$

- Overall attack problem:

$$\begin{aligned} \min_{D \in \mathbb{D}, \mathbf{w}} \quad & \frac{1}{2} \|\mathbf{w} - \mathbf{w}^*\|_2^2 + \frac{\lambda}{2} \|\mathbf{X} - \mathbf{X}_0\|_F^2 \\ \text{s.t.} \quad & w_j - \alpha_i \sum_i \mathbb{I}_1(1 - y_i(\mathbf{x}_i^\top \mathbf{w} + b) \geq 0) y_i x_{ij} = 0. \end{aligned}$$

Using Machine Teaching to Attack Training Set

□ Attacking SVM

- Gradient calculation:

$$\nabla_{\mathbf{X}} = \nabla_{\mathbf{w}} R_A(\mathbf{w}) \Big|_{\hat{\mathbf{w}}(\mathbf{X})} \frac{\partial \hat{\mathbf{w}}(\mathbf{X})}{\partial \mathbf{X}} + \nabla_{\mathbf{X}} E_A(D, D_0)$$

- Risk gradient and effort gradient are:

$$\begin{aligned} \nabla_{\mathbf{w}} R_A(\mathbf{w}) &= \mathbf{w} - \mathbf{w}^* \\ \nabla_{x_{ij}} E_A(D, D_0) &= \lambda(X_{ij} - X_{0,ij}) \end{aligned}$$

- To calculate $\frac{\partial \hat{\mathbf{w}}(\mathbf{X})}{\partial \mathbf{X}}$, the Jacobian matrix is:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{w}} = I.$$

- The Jacobian at row $\mathbf{j}'$ and column $\mathbf{ij}$ is:

$$\left[\frac{\partial \mathbf{f}}{\partial \mathbf{X}} \right]_{j', ij} = -\alpha_i y_i \mathbb{I}_1(j = j') \mathbb{I}_1(1 - y_i \mathbf{x}_i^T \mathbf{w} \geq 0)$$

- The element at row $\mathbf{j}'$ and column $\mathbf{ij}$ in $\frac{\partial \hat{\mathbf{w}}(\mathbf{X})}{\partial \mathbf{X}}$ is:

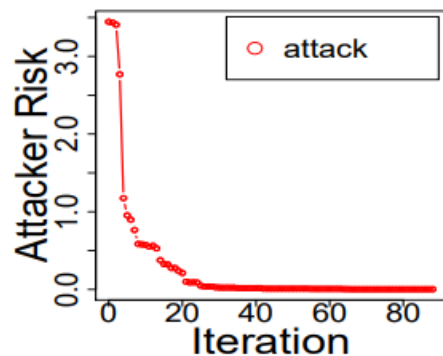
$$\left[\frac{\partial \hat{\mathbf{w}}(\mathbf{X})}{\partial \mathbf{X}} \right]_{j', ij} = \alpha_i y_i \mathbb{I}_1(j = j') \mathbb{I}_1(1 - y_i \mathbf{x}_i^T \mathbf{w} \geq 0)$$

Modifying point $\mathbf{x}$ has effect on $\mathbf{w}$ only if $\mathbf{x}$ has incurred the hinge loss (i.e., $\mathbf{x}$ is a support vector)

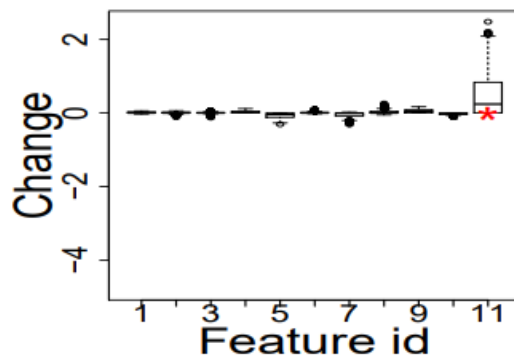
Using Machine Teaching to Attack Training Set

□ Attacking SVM

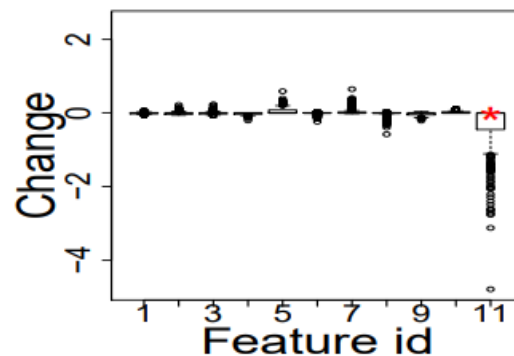
- **Experiments setting:** $n = 1600$, $d = 11$, binary classification of wine quality (y).
- **Attacker goal:** make it look like only the feature ‘alcohol’ correlates with wine quality.
- **How to generate the target weight vector w^* ?** Create another set of labels y' for each data point in D_0 , by threshold the feature ‘alcohol’ > 0 , and then learn the w^* .



(a) attacker risk R_A



(b) feature changes on positive data



(c) feature changes on negative data

Using Machine Teaching to Attack Training Set

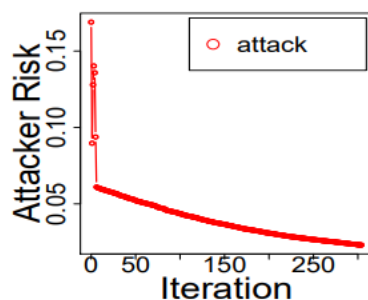
□ Attacking Logistic Regression

- Original data set: $D_0 = (\mathbf{X}_0, \mathbf{y}_0)$
- Attack search space: $\mathbb{D} = \{(\mathbf{X}, \mathbf{y}_0) \mid \mathbf{X} \in \mathbb{R}^{d \times n}\}$
- Learner (lower-level problem):

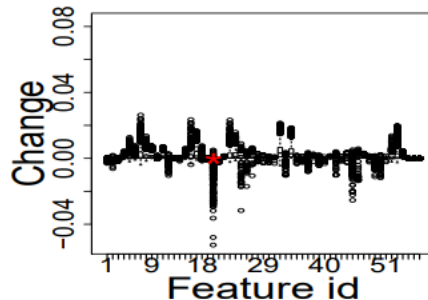
$$O_L(D, \mathbf{w}, b) = \sum_i \log(1 + \exp(-y_i \hat{h}_i)) + \frac{\mu}{2} \|\mathbf{w}\|_2^2$$

- Overall attack problem:

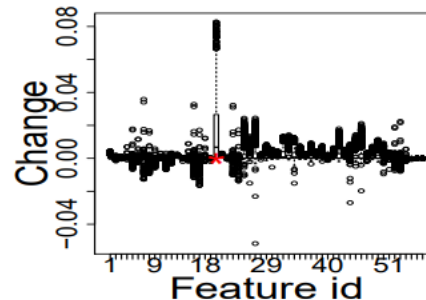
$$\begin{aligned} \min_{D \in \mathbb{D}, \mathbf{w}} \quad & \frac{1}{2} \|\mathbf{w} - \mathbf{w}^*\|_2^2 + \frac{\lambda}{2} \|\mathbf{X} - \mathbf{X}_0\|_F^2 \\ \text{s.t.} \quad & \sum_i -(1 - \sigma(y_i \hat{h}_i)) y_i x_{ij} + \mu w_j = 0 \end{aligned}$$



(a) attacker risk R_A



(b) feature changes on positive data



(c) feature changes on negative data

Using Machine Teaching to Attack Training Set

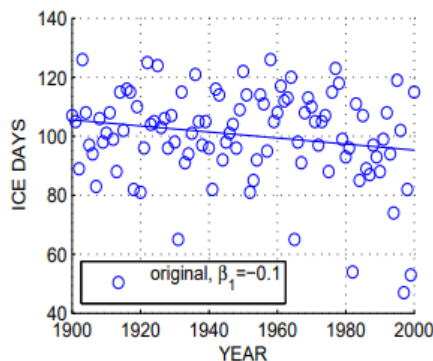
□ Attacking Linear Regression

- Original data set: $D_0 = (\mathbf{X}_0, \mathbf{y}_0)$
- Attack search space: $\mathbb{D} = \{(\mathbf{X}_0, \mathbf{y}_0 + \delta) : \delta \in \mathbb{R}^n\}$
- Learner (lower-level problem):

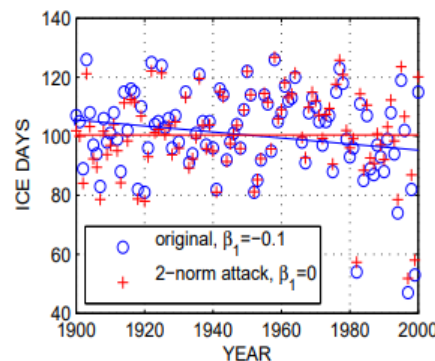
$$O_L(D, \beta) = \|\mathbf{y} - X\beta\|^2$$

- Overall attack problem:

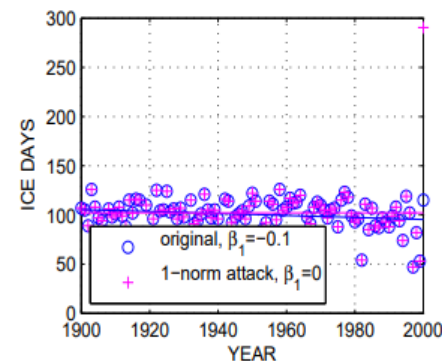
$$\min_{\delta} \|\delta\|_2^2 \quad \text{s.t. } \hat{\beta}_{D,1} \leq 0, \quad \min_{\delta} \|\delta\|_1 \quad \text{s.t. } \hat{\beta}_{D,1} \leq 0$$



OLS on D_0



ℓ_2 attacker effort



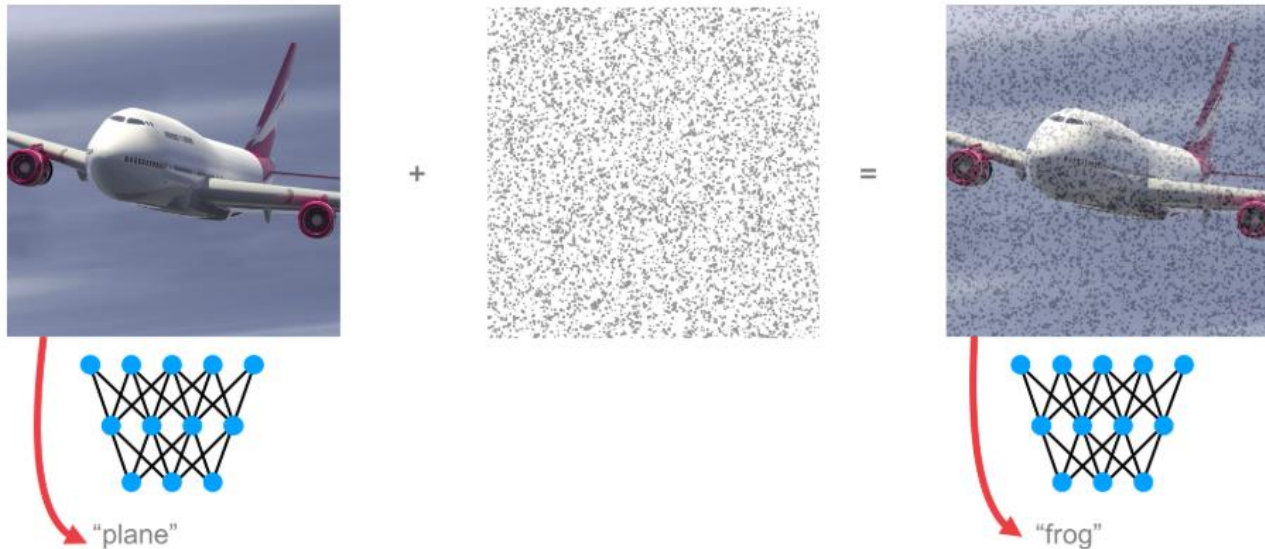
ℓ_1 attacker effort

Figure 3: Training-set attack on OLS

Targeted Clean-Label attack on Neural Networks

□ Attacks on machine learning models

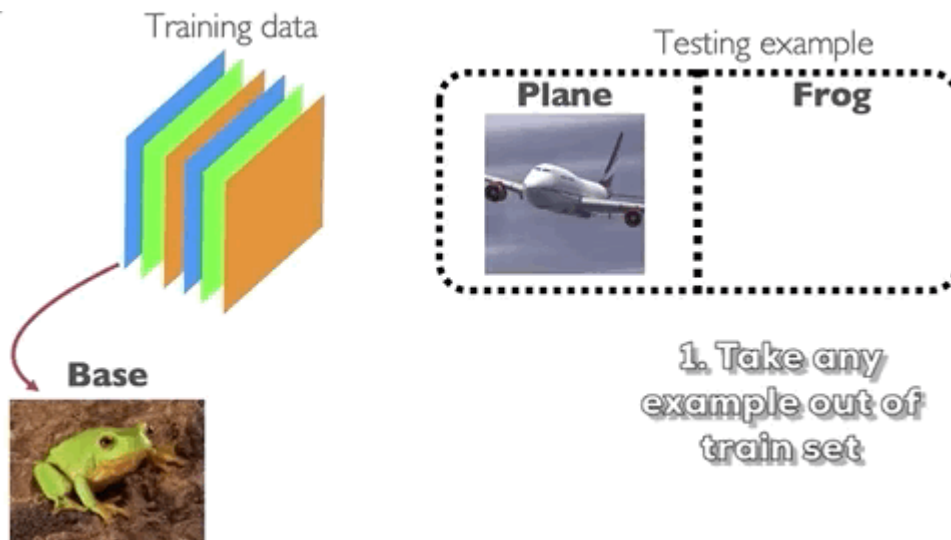
- **Evasion attack:** it is a “test time” attack where the attacker applies the perturbation during test time.



Targeted Clean-Label attack on Neural Networks

□ Attacks on machine learning models

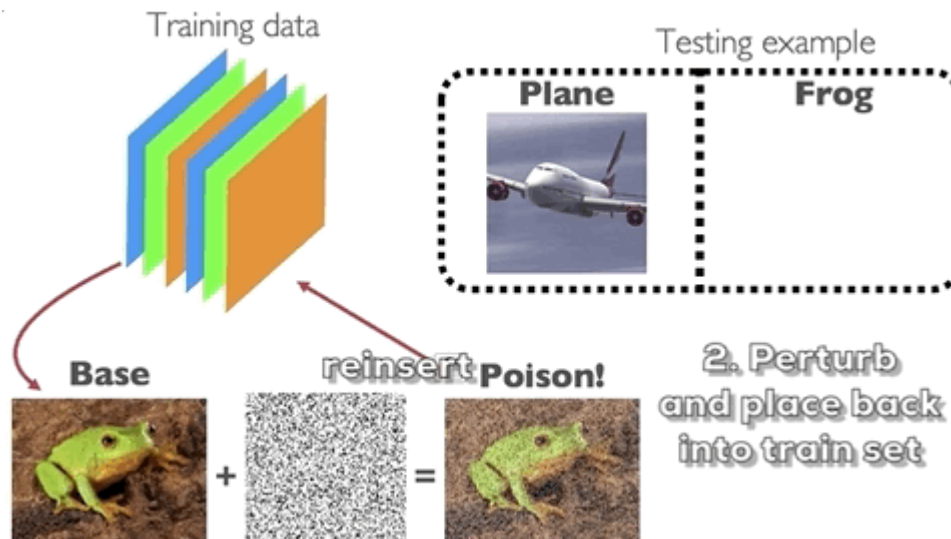
- **Poison attack:** it is a “training set” attack that can manipulate the behavior of the model at test time. Below is the example of clean-label poison attack.



Targeted Clean-Label attack on Neural Networks

□ Attacks on machine learning models

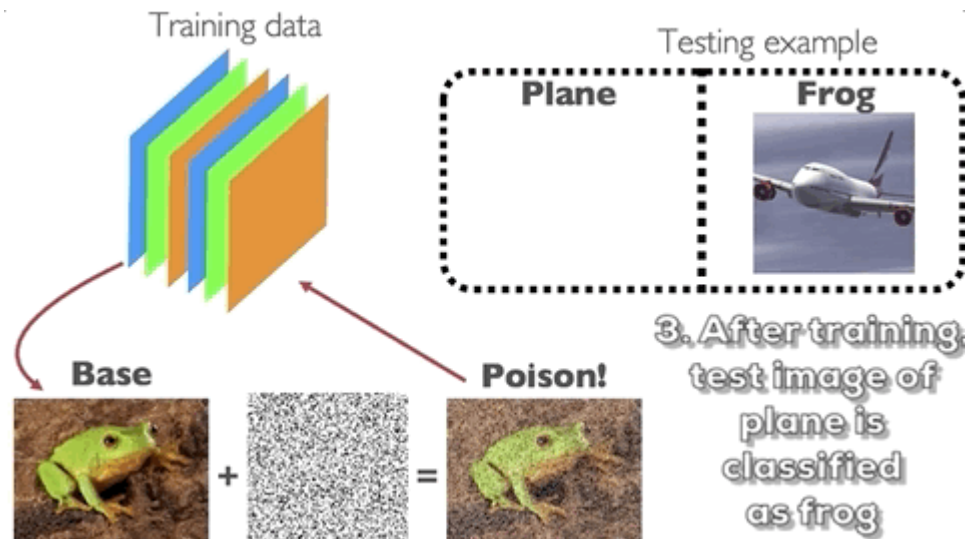
- **Poison attack:** it is a “training set” attack that can manipulate the behavior of the model at test time. Below is the example of clean-label poison attack.



Targeted Clean-Label attack on Neural Networks

□ Attacks on machine learning models

- **Poison attack:** it is a “training set” attack that can manipulate the behavior of the model at test time. Below is the example of *clean-label* poison attack.



Targeted Clean-Label attack on Neural Networks

□ How to craft the poisoning sample?

- The optimization objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

Where, $\mathbf{b}$ is the base instance, $\mathbf{t}$ is the target instance, $\mathbf{p}$ is the created poison instance, $f(\cdot)$ is the output of the neural network on penultimate layer.

Algorithm 1 Poisoning Example Generation

Input: target instance t , base instance b , learning rate λ

Initialize x : $x_0 \leftarrow b$

Define: $L_p(x) = \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2$

for $i = 1$ **to** $maxIters$ **do**

 Forward step: $\hat{x}_i = x_{i-1} - \lambda \nabla_x L_p(x_{i-1})$

 Backward step: $x_i = (\hat{x}_i + \lambda \beta b) / (1 + \beta \lambda)$

end for

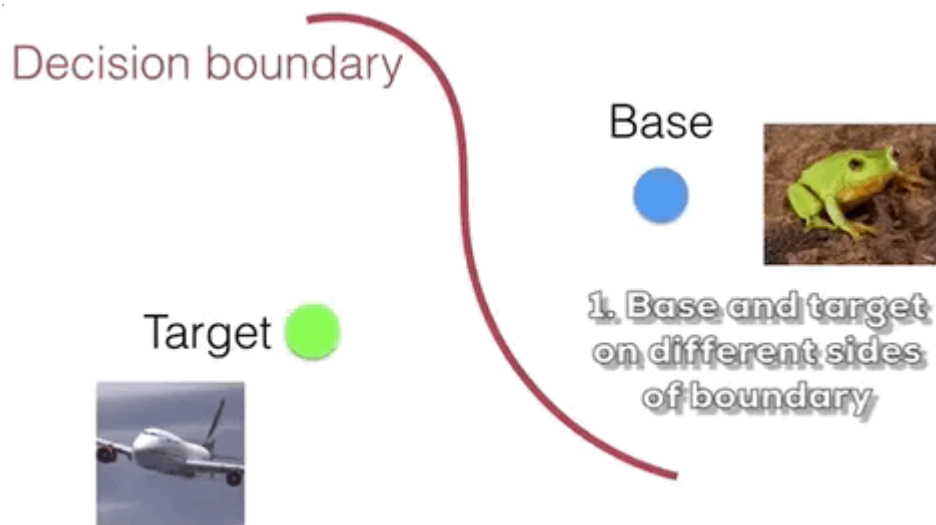
Targeted Clean-Label attack on Neural Networks

□ How to craft the poisoning sample?

- The optimization objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

Where, $\mathbf{b}$ is the base instance, $\mathbf{t}$ is the target instance, $\mathbf{p}$ is the created poison instance, $f(\cdot)$ is the output of the neural network on penultimate layer.



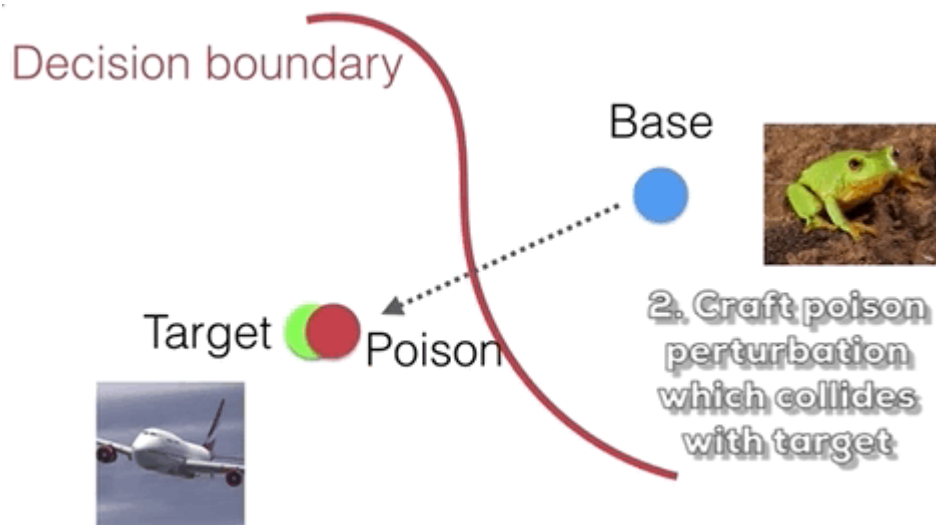
Targeted Clean-Label attack on Neural Networks

□ How to craft the poisoning sample?

- The optimization objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

Where, $\mathbf{b}$ is the base instance, $\mathbf{t}$ is the target instance, $\mathbf{p}$ is the created poison instance, $f(\cdot)$ is the output of the neural network on penultimate layer.



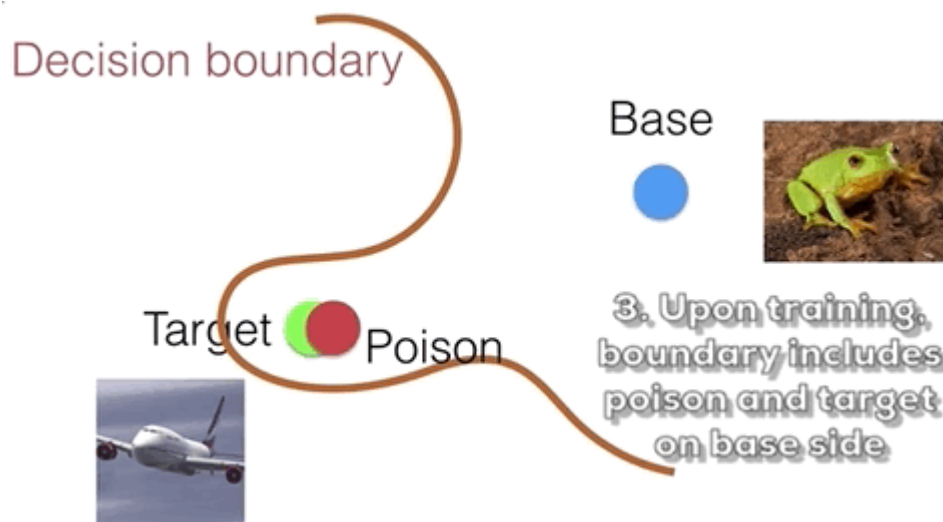
Targeted Clean-Label attack on Neural Networks

□ How to craft the poisoning sample?

- The optimization objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

Where, $\mathbf{b}$ is the base instance, $\mathbf{t}$ is the target instance, $\mathbf{p}$ is the created poison instance, $f(\cdot)$ is the output of the neural network on penultimate layer.

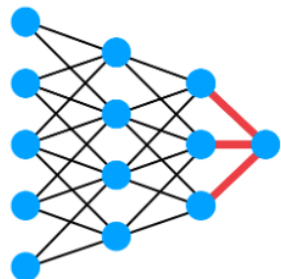


Targeted Clean-Label attack on Neural Networks

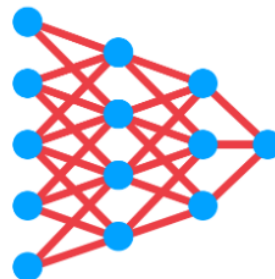
□ Experimental results

- **Transfer learning:** feature extract layers (the layers before the penultimate layer) are frozen, only trained on the final layer.
- **End-to-end re-training:** all weights are trained in the network.

Transfer learning



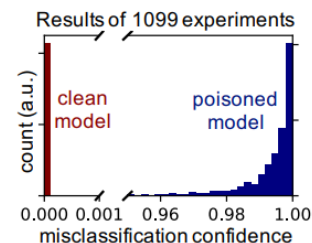
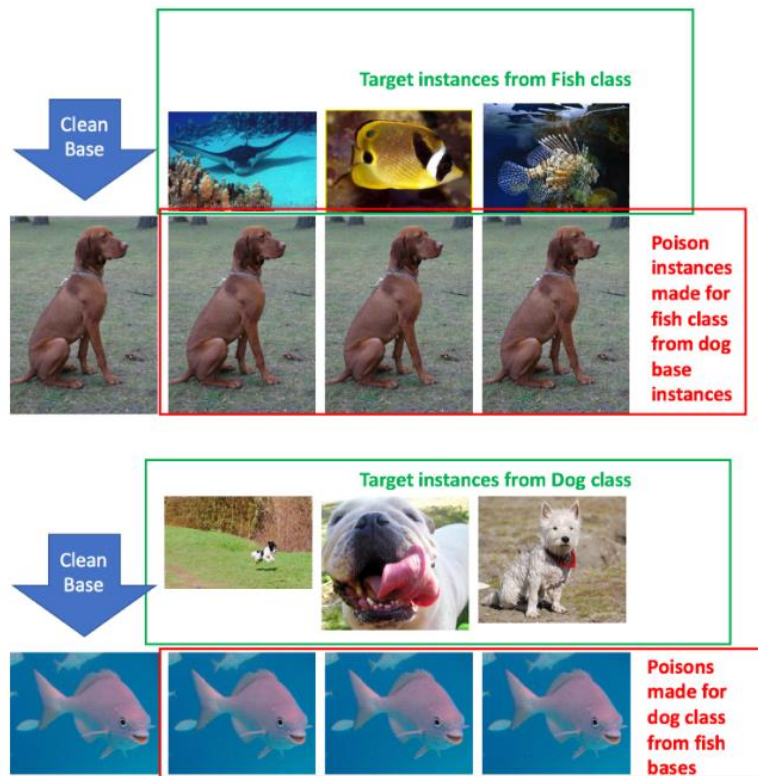
End-to end re-training



Targeted Clean-Label attack on Neural Networks

□ Transfer learning (one-shot kill attack)

- dog vs. fish with 1099 test instance.
- Success rate of 100%.

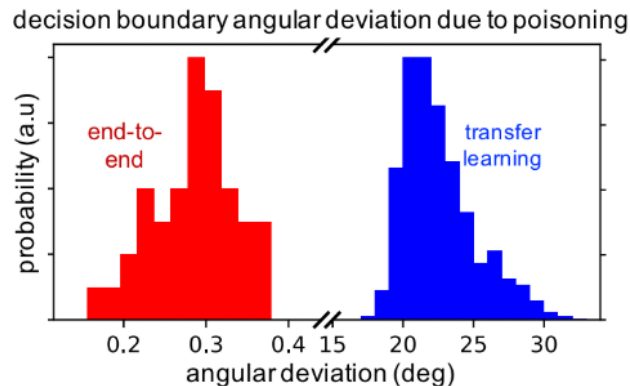


Incorrect class's probability histogram predicted for the target image by the clean (dark red) and poisoned (dark blue) models. When trained on a poisoned dataset, the target instances not only get misclassified; they get misclassified with high confidence.

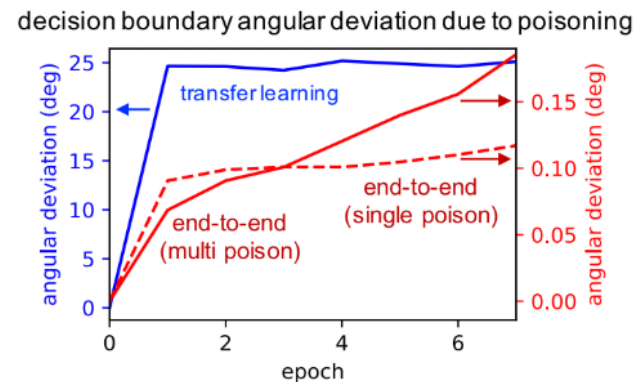
Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

- Not very effective, comparing with transfer learning.
- Angular deviation = angular difference between weight vectors in clean and poisoned models



(a) PDF of decision boundary angular deviation.

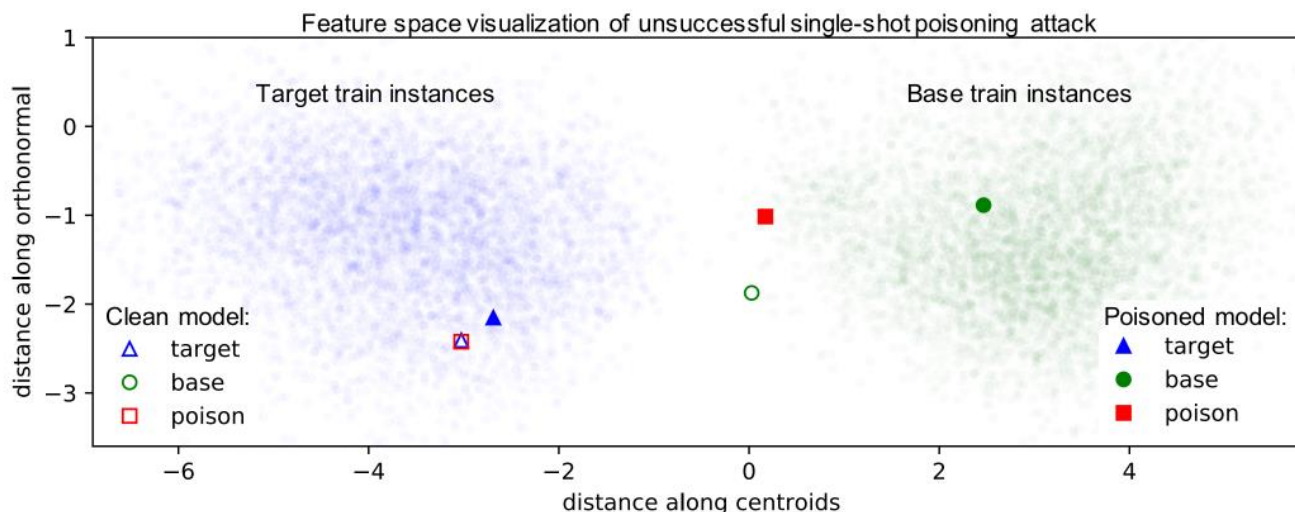


(b) Average angular deviation vs epoch.

Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

- **Feature space visualization:** successful attack will have the target and poison overlapped in the feature space.
- **Observation and reason:** during retraining with the poison data, the network modifies its lower-level feature extraction kernels in the shallow layers so the poison instance is returned to the base class distribution in the deep layers



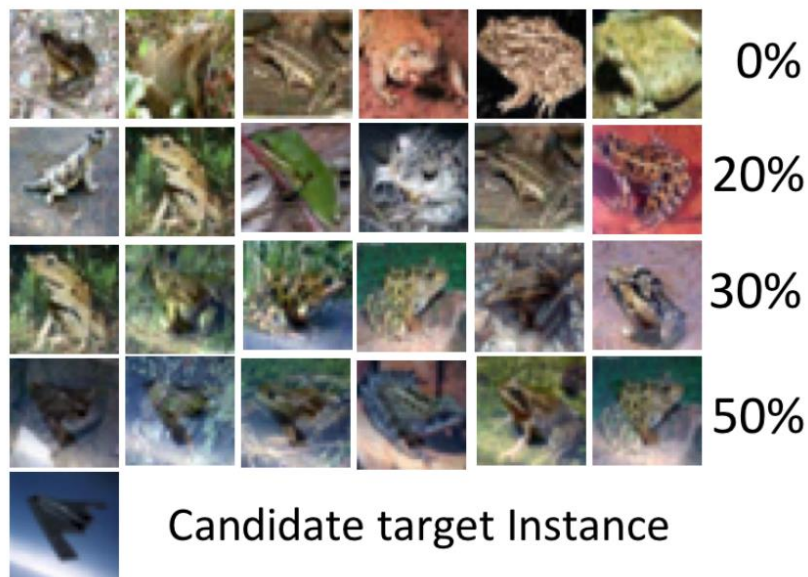
Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

- **Watermark:** Watermarking blends additional features of the target instance into the poison instance in a way humans can't notice. Watermarks are not noticeable to humans up to 30% opacity.

$$\mathbf{b} \leftarrow \gamma \cdot \mathbf{t} + (1 - \gamma) \cdot \mathbf{b}$$

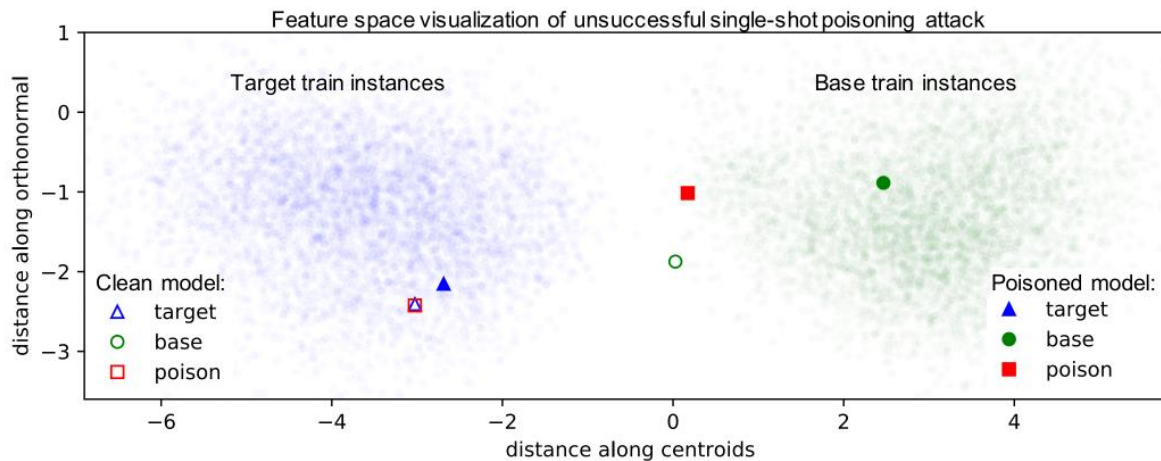
- **Multi-poison attack:** create multiple poison instance.



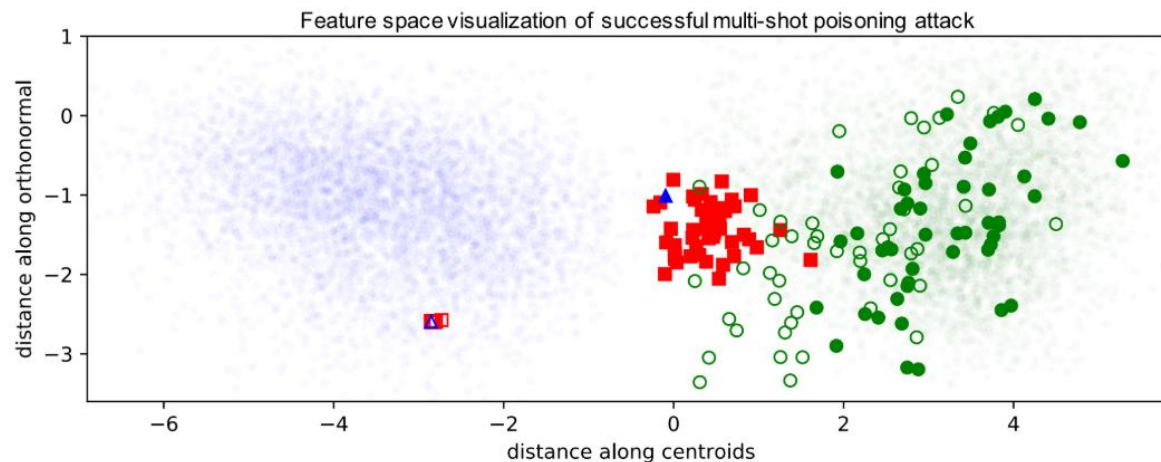
Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

○ Feature space visualization.



Single attack

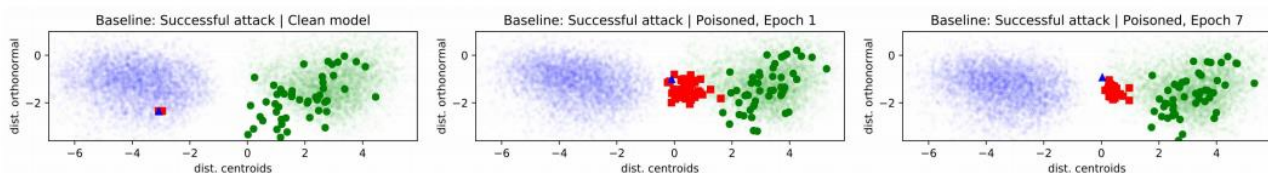


Multiple
watermarked
attacks with
30% opacity

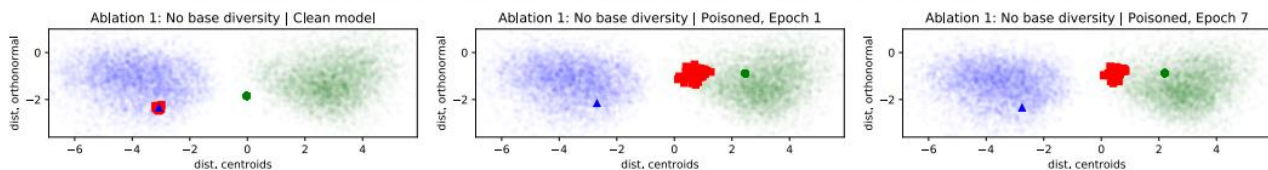
Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

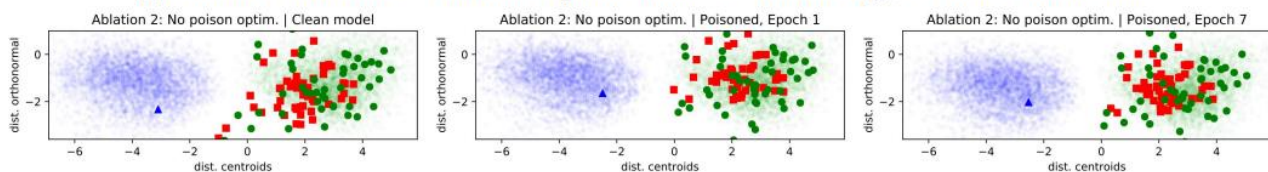
○ Ablation study:



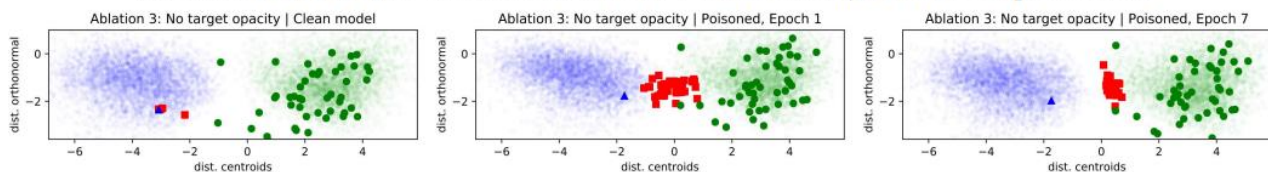
(a) Successful attack - includes: optimization + watermarking + multi-base



(b) Unsuccessful attack - includes: optimization + watermarking ; excludes: multi-base



(c) Unsuccessful attack - includes: multi-base + watermarking ; excludes: optimization



(d) Unsuccessful attack - includes: optimization + multi-base ; excludes: watermarking

Targeted Clean-Label attack on Neural Networks

- End-to-end attack
 - More experiments:



Figure 10: Some sample target and poison instances for the task of attacking a target instance from the bird class using base instances from the dog class. The columns with the green dot above them are successful attacks. Similar, to the other experiment (attack), a 30% opacity of the target is added to the base instance. Note that the poison instances here are better looking than the poison instances of the plane-frog task. Also, the attacks are more successful for this task. To verify that this attack is hardly detectable to a non-suspicious label auditor, one should only look at one column and ignore the top row because the auditor/labeler is not aware of the target.

Targeted Clean-Label attack on Neural Networks

□ End-to-end attack

- **More experiments:**



Figure 9: Some samples of poisons made for different plane targets using base instances as frogs. Note that, the bases were watermarked with a 30% opacity of the target. The target instance is on the top row and some of the poisons are below it. A green dot is used when the attack was successful and red dots indicate unsuccessful attacks. Note that the images do not always look clean compared to the bird vs dog task presented in Fig. 10. But it is important to note that when the target is not known, it may be hard to recognize its existence in all images.